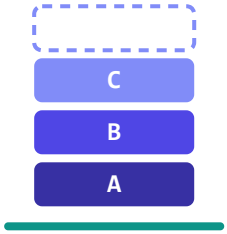
LAPORAN TUGAS
MATA KULIAH STRUKTUR DATA

STRUKTUR DATA PLAYGROUND

Aplikasi Web Visualisasi Interaktif
Stack (LIFO) dan Queue (FIFO)



Disusun oleh:

Kenny Ramadhan	312510195
Ariq Firmansyah	312510176
Muhammad Ryno Al Faridz	312510022
Dwi Afandi	312510100

MATA KULIAH STRUKTUR DATA
2026

DAFTAR ISI

BAB I PENDAHULUAN

1.1 Latar Belakang

1.2 Tujuan

1.3 Ruang Lingkup

1.4 Teknologi yang Digunakan

BAB II LANDASAN TEORI

2.1 Stack (LIFO)

2.2 Queue (FIFO)

2.3 Circular Queue

2.4 Perbandingan Stack dan Queue

BAB III PERANCANGAN SISTEM

3.1 Arsitektur Aplikasi

3.2 Perancangan Basis Data

3.3 Pembatasan Login Empat Anggota

3.4 Implementasi Struktur Data

BAB IV HASIL DAN PEMBAHASAN

4.1 Halaman Login

4.2 Menu Utama

4.3 Menu Stack (LIFO)

4.4 Menu Queue (FIFO)

4.5 Menu FIFO vs LIFO

4.6 Riwayat Operasi, Logout, dan Tampilan Responsif

4.7 Pengujian

BAB V PENUTUP

5.1 Kesimpulan

5.2 Saran

LAMPIRAN

A. Cara Menjalankan Aplikasi

B. Akun Login

BAB I
PENDAHULUAN

1.1 Latar Belakang

Struktur data merupakan cara mengatur dan menyimpan data di dalam komputer agar dapat diolah secara efisien. Dua struktur data linear yang paling mendasar adalah *stack* (tumpukan) dan *queue* (antrian). Keduanya menyimpan data secara berurutan, tetapi berbeda dalam aturan pengambilan data: *stack* mengikuti prinsip LIFO (*Last In, First Out*), sedangkan *queue* mengikuti prinsip FIFO (*First In, First Out*).

Konsep tersebut sering sulit dibayangkan jika hanya dipelajari melalui teori dan potongan kode. Oleh karena itu, kelompok kami membangun aplikasi web bernama **Struktur Data Playground** yang memvisualisasikan cara kerja *stack* dan *queue* secara interaktif. Pengguna dapat menjalankan setiap operasi, melihat perubahan isi struktur data dan penunjuknya secara langsung, serta mengamati penerapannya pada kasus nyata dalam bentuk animasi langkah demi langkah.

1.2 Tujuan

1. Mengimplementasikan struktur data *stack* dan *queue* secara manual (tanpa memakai fungsi bawaan `Array.push()` / `Array.pop()`) dalam bahasa JavaScript.
2. Memvisualisasikan prinsip LIFO dan FIFO beserta kondisi *overflow* dan *underflow*.
3. Menunjukkan penerapan *stack* dan *queue* pada permasalahan nyata.
4. Membangun aplikasi web lengkap dengan sistem login yang hanya dapat diakses oleh empat anggota kelompok, serta basis data SQLite.

1.3 Ruang Lingkup

1. Struktur data yang dibahas adalah *stack* berbasis *array* statis dan *circular queue*, masing-masing berkapasitas 8 elemen.
2. Penerapan yang ditampilkan: pengecekan tanda kurung seimbang, pembalikan kata, simulasi antrian loket, dan perbandingan FIFO dengan LIFO.
3. Aplikasi dijalankan secara lokal, baik menggunakan Docker maupun Node.js.

1.4 Teknologi yang Digunakan

Tabel 1.1 Teknologi yang digunakan

Komponen	Teknologi	Kegunaan
Frontend	React 19 + Vite	Antarmuka pengguna dan visualisasi
Backend	Node.js + Express 5	API, autentikasi, dan logika struktur data
Basis data	SQLite (`node:sqlite`)	Menyimpan akun, isi struktur data, dan riwayat operasi
Autentikasi	JWT + bcrypt	Token sesi login dan *hash* kata sandi
Pengujian	`node:test`	Unit test struktur data
Deployment	Docker + Docker Compose	Menjalankan aplikasi dengan satu perintah
Repositori	GitHub	github.com/kennyRamadhan/struktur_data

BAB II
LANDASAN TEORI

2.1 Stack (LIFO)

Stack atau tumpukan adalah struktur data linear yang hanya mengizinkan penambahan dan pengambilan data pada satu ujung yang disebut **TOP**. Prinsip yang digunakan adalah LIFO (*Last In, First Out*): data yang terakhir dimasukkan akan menjadi data pertama yang dikeluarkan. Analogi sehari-harinya adalah tumpukan piring, karena piring yang diletakkan terakhir berada paling atas dan diambil lebih dahulu.

Tabel 2.1 Operasi pada stack

Operasi	Keterangan	Kompleksitas
`push(x)`	Menambahkan data *x* pada posisi TOP	O(1)
`pop()`	Mengambil dan menghapus data pada posisi TOP	O(1)
`peek()`	Melihat data TOP tanpa menghapusnya	O(1)
`isEmpty()`	Memeriksa apakah *stack* kosong (`top = -1`)	O(1)
`isFull()`	Memeriksa apakah *stack* penuh (`top = kapasitas - 1`)	O(1)

Kondisi ***overflow*** terjadi ketika `push` dilakukan pada *stack* yang sudah penuh, sedangkan ***underflow*** terjadi ketika `pop` atau `peek` dilakukan pada *stack* yang kosong.

2.2 Queue (FIFO)

Queue atau antrian adalah struktur data linear yang menambahkan data pada ujung belakang (**REAR**) dan mengambil data dari ujung depan (**FRONT**). Prinsip yang digunakan adalah FIFO (*First In, First Out*): data yang pertama masuk adalah data yang pertama keluar, seperti antrian di loket bank.

Tabel 2.2 Operasi pada queue

Operasi	Keterangan	Kompleksitas
`enqueue(x)`	Menambahkan data *x* di posisi REAR	O(1)
`dequeue()`	Mengambil dan menghapus data di posisi FRONT	O(1)
`peek()`	Melihat data FRONT tanpa menghapusnya	O(1)
`isEmpty()` / `isFull()`	Memeriksa apakah *queue* kosong atau penuh	O(1)

2.3 Circular Queue

Pada *queue* linear berbasis *array*, setiap operasi `dequeue` menggeser penunjuk FRONT ke kanan sehingga slot di depannya kosong tetapi tidak dapat dipakai kembali. Akibatnya *queue* dapat dianggap penuh padahal masih memiliki slot kosong. *Circular queue* mengatasi masalah ini dengan menganggap ujung *array* tersambung kembali ke awal *array*. Penunjuk digerakkan menggunakan operasi modulo:

```
rear  = (rear  + 1) % kapasitas
front = (front + 1) % kapasitas
```

Dengan cara ini, slot yang sudah dikosongkan dapat digunakan kembali dan seluruh operasi tetap berjalan dalam waktu O(1). Variabel `count` dipakai untuk membedakan kondisi kosong dan penuh.

2.4 Perbandingan Stack dan Queue

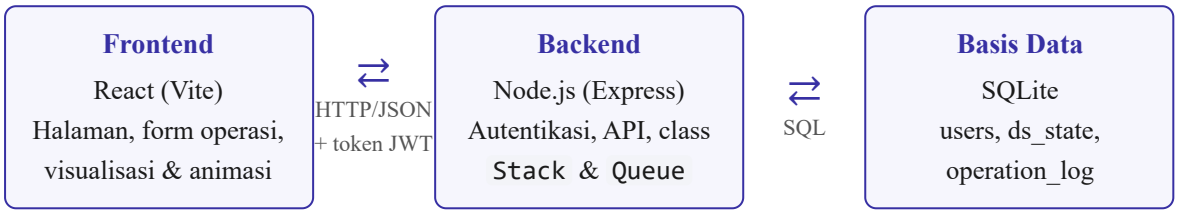
Tabel 2.3 Perbandingan stack dan queue

Aspek	Stack	Queue
Prinsip	LIFO	FIFO
Ujung yang digunakan	Satu ujung (TOP)	Dua ujung (FRONT dan REAR)
Urutan keluar	Kebalikan urutan masuk	Sama dengan urutan masuk
Contoh penerapan	Tombol *undo*, tombol *back* pada peramban, pengecekan tanda kurung	Antrian loket, antrian cetak printer, penjadwalan proses CPU

BAB III
PERANCANGAN SISTEM

3.1 Arsitektur Aplikasi

Aplikasi menggunakan arsitektur *client-server*. Antarmuka React mengirim permintaan HTTP ke server Node.js. Server menjalankan logika struktur data, menyimpan hasilnya ke SQLite, lalu mengirim kembali state terbaru dalam format JSON untuk divisualisasikan.



Gambar 3.1 Arsitektur aplikasi

Logika struktur data sengaja ditempatkan di sisi server (folder `server/src/ds/`) agar implementasinya terpusat, mudah diuji, dan isi struktur data dapat disimpan untuk setiap pengguna. Alur sebuah operasi, misalnya *push*, adalah sebagai berikut:

1. Pengguna menekan tombol **Push**, lalu React mengirim `POST /api/stack/push` beserta token JWT.
2. Server memverifikasi token, lalu memuat state *stack* milik pengguna dari SQLite.
3. Method `stack.push(nilai)` dijalankan. Jika *stack* penuh, terjadi *overflow*.
4. State baru disimpan ke tabel `ds_state` dan operasi dicatat ke tabel `operation_log`.
5. Server mengirim state terbaru, lalu React menggambar ulang visualisasinya.

3.2 Perancangan Basis Data

Tabel 3.1 Struktur tabel basis data

Tabel	Kolom	Keterangan
`users`	id, username, password_hash, name, nim	Akun empat anggota kelompok. Kata sandi disimpan sebagai *hash* bcrypt.
`ds_state`	user_id, type, state, updated_at	Isi *stack*/*queue* terakhir milik setiap pengguna dalam format JSON.
`operation_log`	id, user_id, type, operation, value, success, message, created_at	Riwayat seluruh operasi, baik yang berhasil maupun gagal.

3.3 Pembatasan Login Empat Anggota

Sesuai ketentuan tugas, hanya empat anggota kelompok yang dapat masuk ke aplikasi. Pembatasan diterapkan secara berlapis:

1. Tidak tersedia fitur registrasi. Daftar akun hanya berasal dari berkas `server/src/data/members.js`.
2. Setiap server dijalankan, tabel `users` disinkronkan dengan daftar tersebut.
3. Basis data memiliki *trigger* `limit_users` yang menolak penambahan pengguna kelima dengan pesan *"Maksimal 4 user"*.
4. Seluruh halaman selain login dilindungi token JWT yang berlaku selama 8 jam.

```
CREATE TRIGGER IF NOT EXISTS limit_users
BEFORE INSERT ON users
WHEN (SELECT COUNT(*) FROM users) >= 4
BEGIN
  SELECT RAISE(ABORT, 'Maksimal 4 user');
END;
```

3.4 Implementasi Struktur Data

a. Stack (server/src/ds/Stack.js)

Stack diimplementasikan menggunakan *array* berkapasitas tetap dan penunjuk `top` yang bernilai -1 ketika kosong.

```
push(value) {
  if (this.isFull()) {
    throw new DataStructureError(OVERFLOW, 'Stack penuh (overflow), tidak bisa push');
  }
  this.top = this.top + 1;
  this.items[this.top] = value;
  return value;
}

pop() {
  if (this.isEmpty()) {
    throw new DataStructureError(UNDERFLOW, 'Stack kosong (underflow), tidak bisa
pop');
  }
  const value = this.items[this.top];
  this.items[this.top] = null;
  this.top = this.top - 1;
  return value;
}
```

b. Circular Queue (server/src/ds/Queue.js)

Queue diimplementasikan sebagai *circular queue* dengan penunjuk `front`, `rear`, dan penghitung `count`.

```
enqueue(value) {
  if (this.isFull()) {
    throw new DataStructureError(OVERFLOW, 'Queue penuh (overflow), tidak bisa
enqueue');
  }
  this.rear = (this.rear + 1) % this.capacity;
  this.slots[this.rear] = value;
  this.count = this.count + 1;
  return value;
}

dequeue() {
  if (this.isEmpty()) {
    throw new DataStructureError(UNDERFLOW, 'Queue kosong (underflow), tidak bisa
dequeue');
  }
  const value = this.slots[this.front];
  this.slots[this.front] = null;
  this.front = (this.front + 1) % this.capacity;
  this.count = this.count - 1;
  return value;
}
```

c. Algoritma Penerapan

Tabel 3.2 Algoritma penerapan stack dan queue

Berkas / fungsi	Struktur	Cara kerja
`checkBrackets()`	Stack	Kurung buka di-*push*. Kurung tutup mem-*pop* lalu dicocokkan. Ekspresi seimbang jika semua cocok dan *stack* kosong di akhir.
`reverseText()`	Stack	Semua huruf di-*push*, kemudian di-*pop* satu per satu sehingga urutannya terbalik.
`simulateCounter()`	Queue	Pelanggan di-*enqueue* saat datang dan di-*dequeue* saat dilayani sesuai urutan kedatangan.
`compareFifoLifo()`	Stack & Queue	Data yang sama dimasukkan ke keduanya, lalu urutan keluarnya dibandingkan.

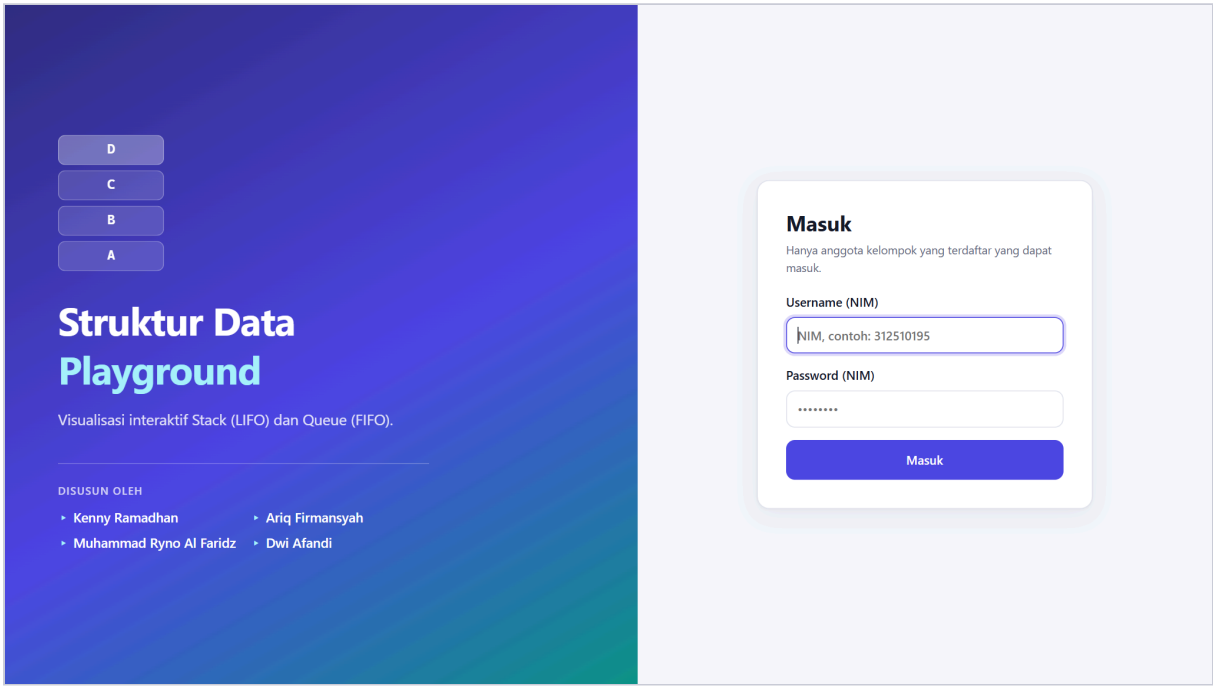
Setiap algoritma mengembalikan daftar langkah (*steps*) berisi kondisi struktur data pada setiap tahap, sehingga antarmuka dapat memutarnya sebagai animasi.

BAB IV
HASIL DAN PEMBAHASAN

Bab ini menyajikan tampilan dan cara kerja aplikasi secara berurutan, dimulai dari halaman login hingga seluruh menu yang tersedia.

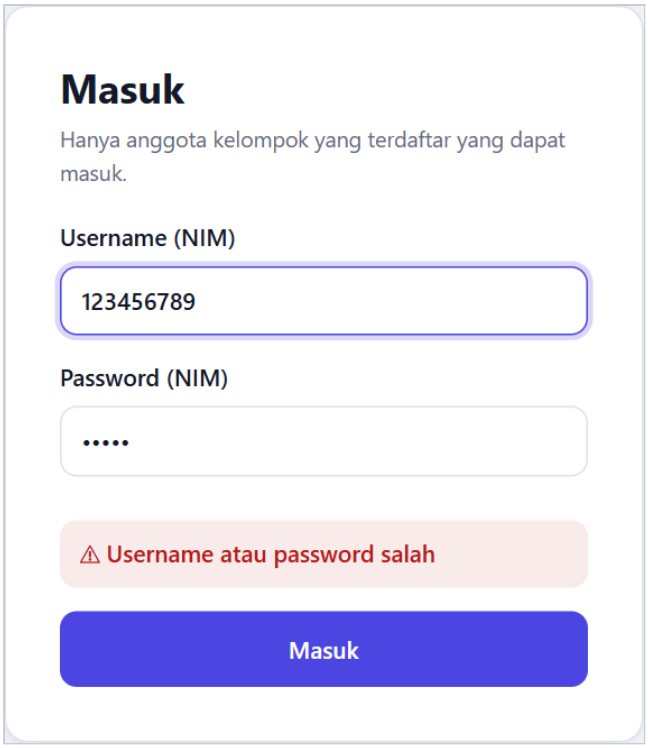
4.1 Halaman Login

Halaman login merupakan halaman pertama yang muncul ketika aplikasi dibuka. Bagian kiri menampilkan judul aplikasi beserta nama anggota kelompok penyusun, sedangkan bagian kanan berisi formulir login. Pengguna masuk menggunakan NIM sebagai *username* dan *password*.



Gambar 4.1 Halaman login

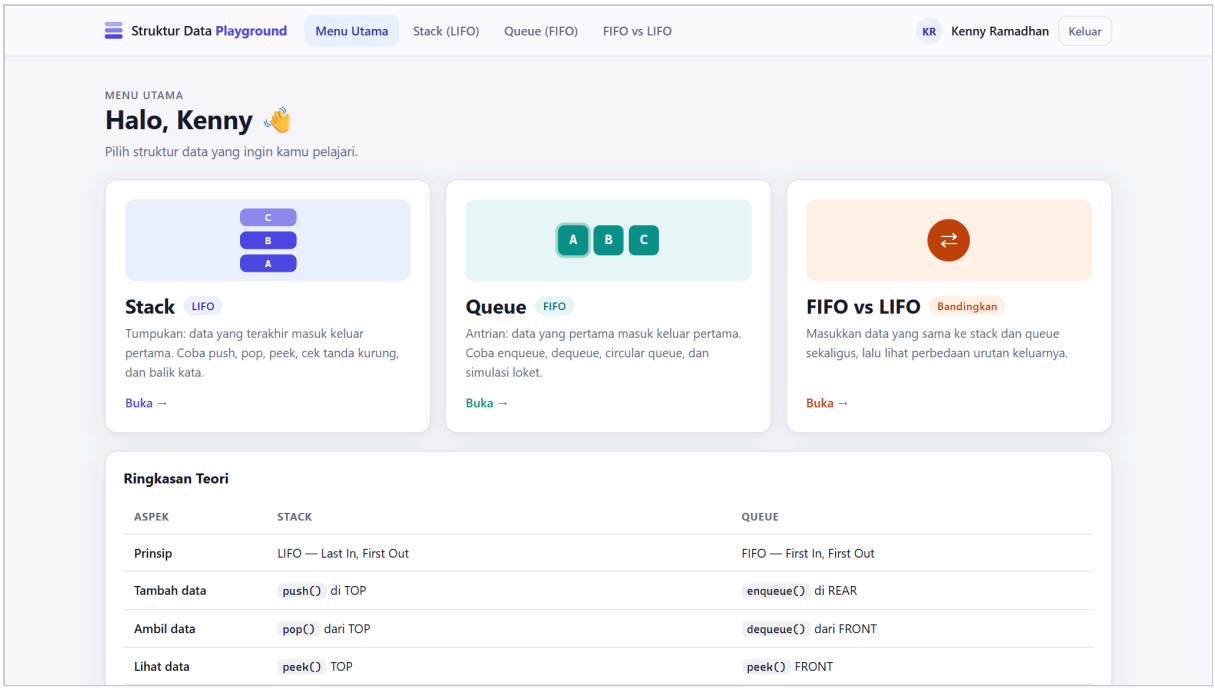
Apabila *username* atau *password* tidak terdaftar, sistem menolak akses dan menampilkan pesan kesalahan. Dengan demikian, selain empat anggota kelompok tidak ada pengguna yang dapat masuk.



Gambar 4.2 Pesan kesalahan ketika login gagal

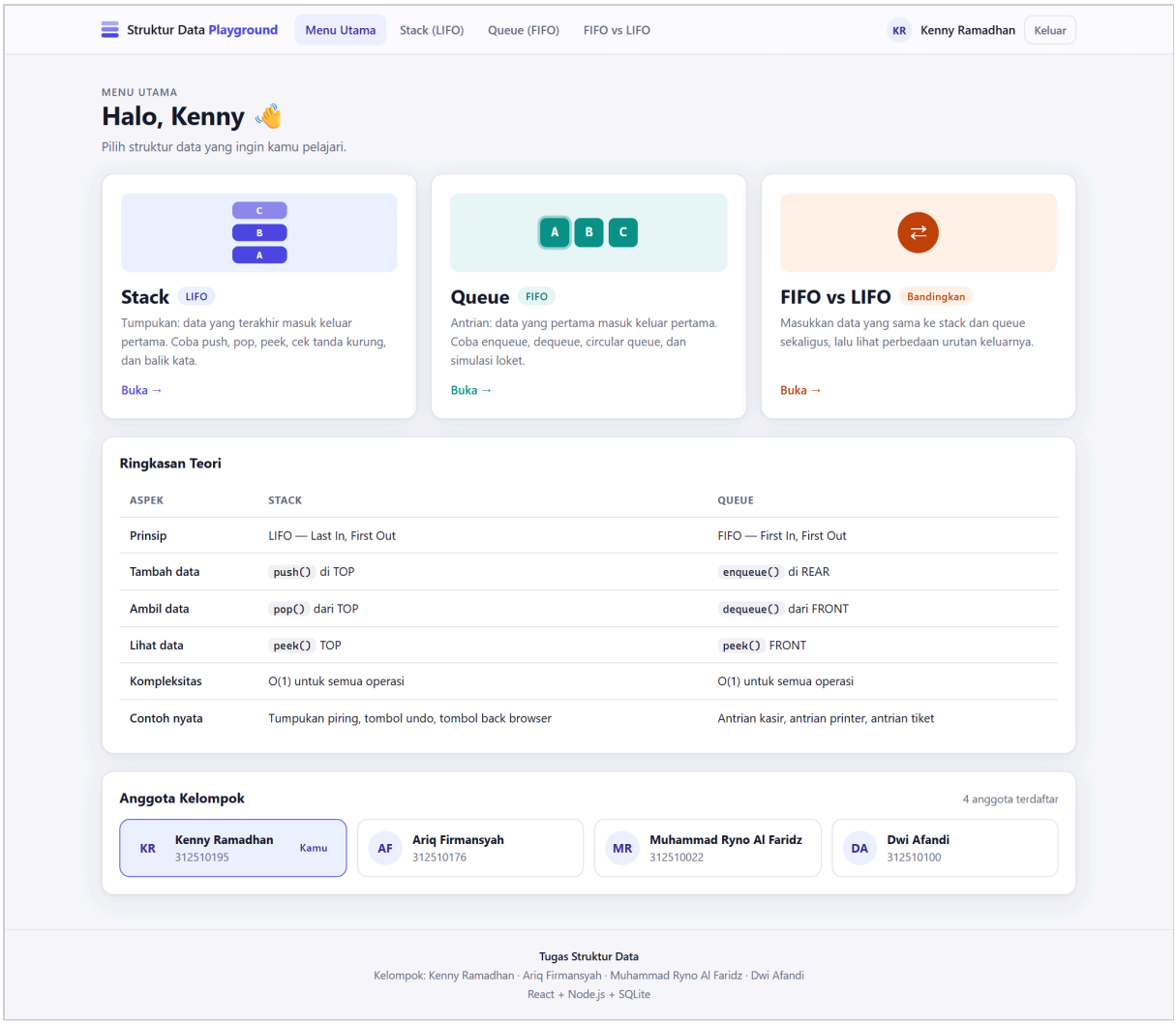
4.2 Menu Utama

Setelah login berhasil, pengguna diarahkan ke Menu Utama. Bagian atas halaman berisi navigasi ke seluruh menu serta nama pengguna yang sedang login dan tombol **Keluar**. Menu Utama menyediakan tiga kartu menu, yaitu **Stack (LIFO)**, **Queue (FIFO)**, dan **FIFO vs LIFO**.



Gambar 4.3 Menu utama setelah login

Di bawah kartu menu terdapat tabel ringkasan teori yang membandingkan *stack* dan *queue*, diikuti daftar empat anggota kelompok beserta NIM. Akun yang sedang login diberi penanda **Kamu**. Bagian *footer* pada setiap halaman juga mencantumkan nama seluruh anggota kelompok.

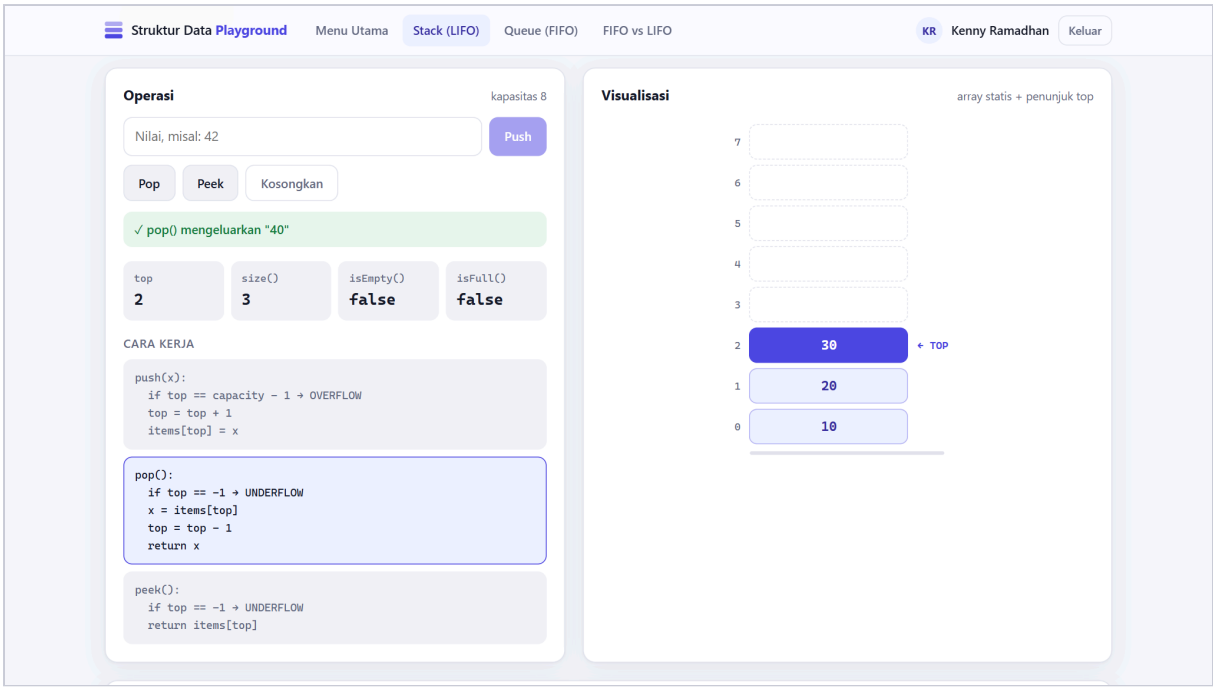


Gambar 4.4 Menu utama lengkap: ringkasan teori, anggota kelompok, dan *footer*

4.3 Menu Stack (LIFO)

a. Operasi push, pop, dan peek

Halaman Stack terbagi menjadi panel **Operasi** di sebelah kiri dan panel **Visualisasi** di sebelah kanan. Pada contoh berikut, data 10, 20, 30, dan 40 di-*push* secara berurutan, kemudian dilakukan satu kali *pop*. Data yang keluar adalah **40**, yaitu data yang terakhir masuk. Hal ini membuktikan prinsip LIFO.

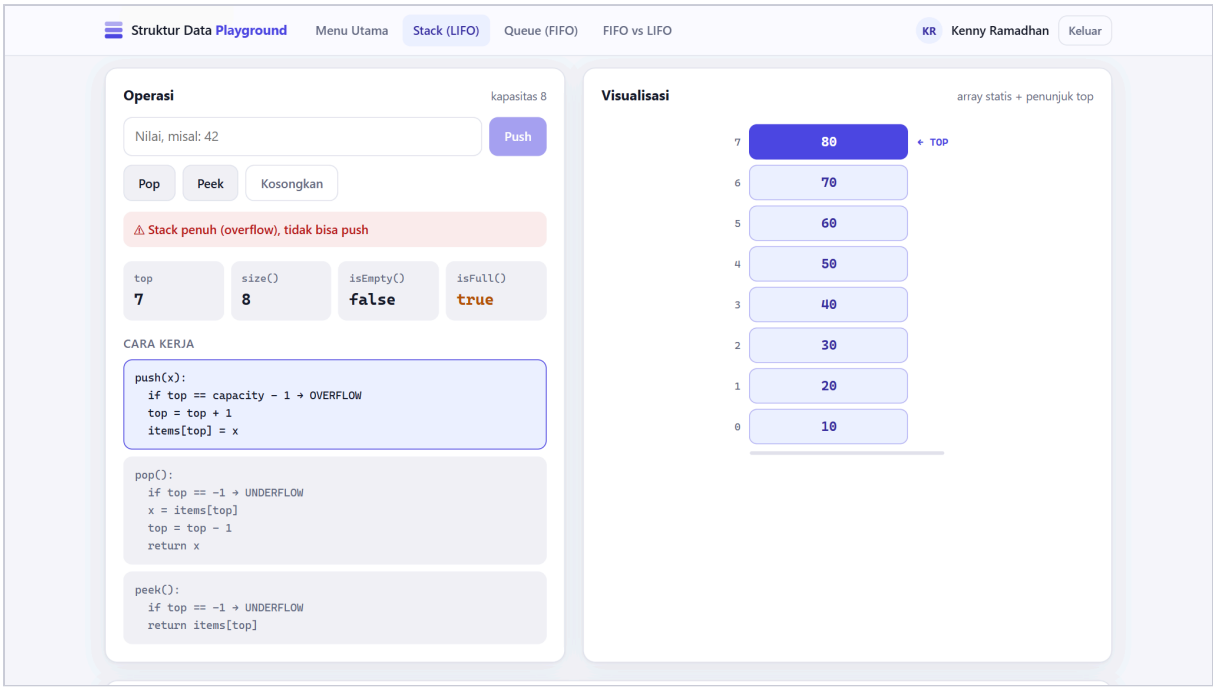


Gambar 4.5 Hasil operasi pop: data terakhir (40) keluar lebih dahulu

Panel visualisasi menampilkan *array* berkapasitas 8 dengan index 0 di bagian bawah. Elemen teratas diberi warna penuh dan label ← **TOP**. Panel operasi menampilkan nilai `top`, `size()`, `isEmpty()`, dan `isFull()`, serta *pseudo-code* **Cara Kerja** yang menyorot operasi terakhir yang dijalankan (pada gambar di atas: `pop()`).

b. Kondisi overflow

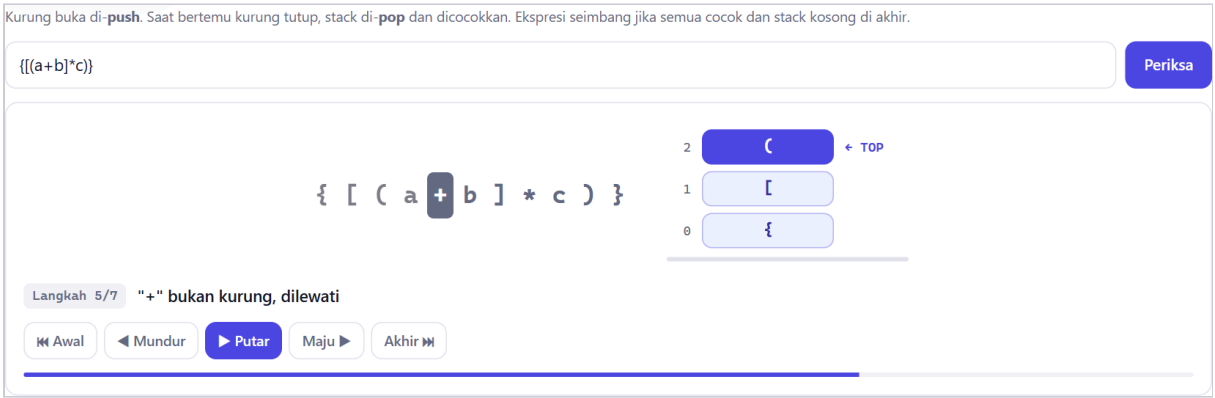
Ketika *stack* telah berisi 8 data (`top = 7`, `isFull() = true`) dan pengguna mencoba melakukan *push* lagi, sistem menolak operasi tersebut dan menampilkan pesan *"Stack penuh (overflow), tidak bisa push"*. Sebaliknya, *pop* pada *stack* kosong menghasilkan pesan *underflow*.



Gambar 4.6 Kondisi overflow ketika stack penuh

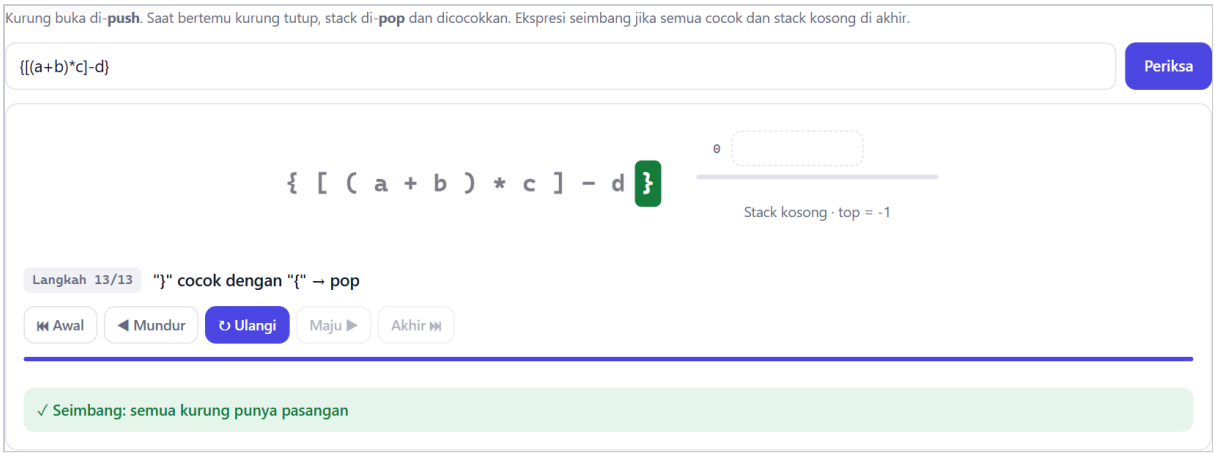
c. Penerapan: cek tanda kurung seimbang

Bagian **Penerapan Stack** menyediakan demo pengecekan tanda kurung. Karakter yang sedang diproses diberi warna: biru untuk *push*, hijau untuk *pop* yang cocok, merah untuk kesalahan, dan abu-abu untuk karakter yang dilewati. Proses dapat diputar otomatis atau dijalankan langkah demi langkah menggunakan tombol **Awal**, **Mundur**, **Putar/Jeda**, **Maju**, dan **Akhir**.



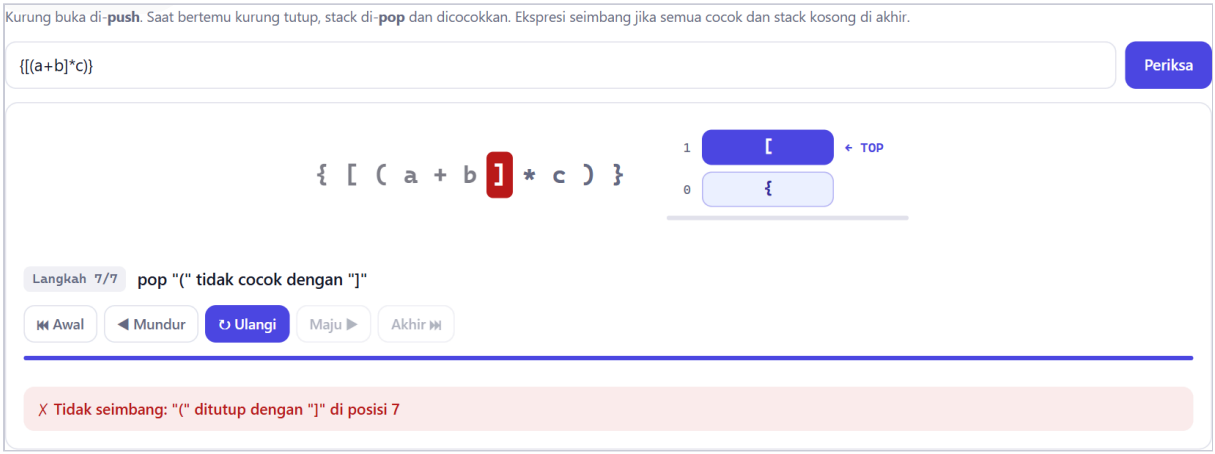
Gambar 4.7 Proses pengecekan: kurung buka `{`, `[`, `(` telah di-push ke stack

Untuk ekspresi `{[(a+b)*c]-d}`, setiap kurung tutup cocok dengan kurung buka di posisi TOP dan *stack* kosong di akhir proses, sehingga ekspresi dinyatakan **seimbang**.



Gambar 4.8 Hasil pengecekan ekspresi seimbang

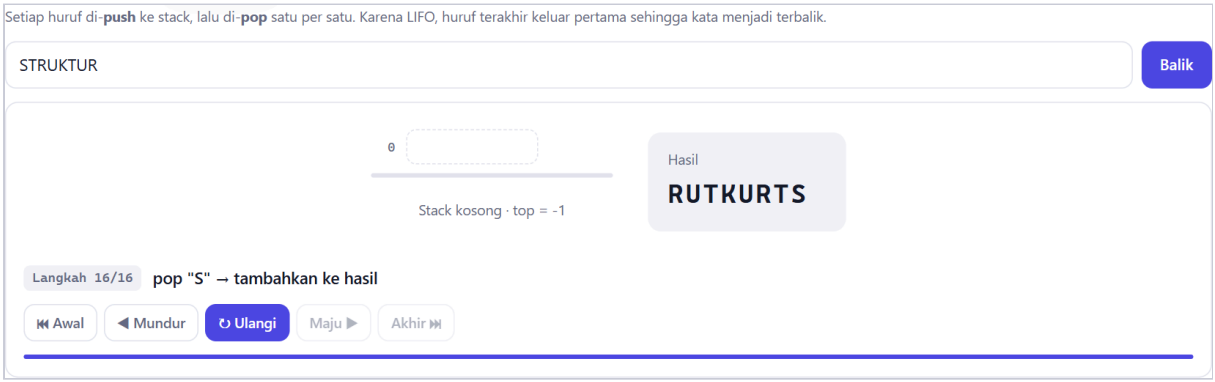
Untuk ekspresi `{[(a+b]*c)}`, ketika karakter `]` diproses, data yang di-*pop* adalah `(`. Keduanya bukan pasangan, sehingga ekspresi dinyatakan **tidak seimbang**.



Gambar 4.9 Hasil pengecekan ekspresi tidak seimbang

d. Penerapan: balik kata

Pada tab **Balik Kata**, setiap huruf dari kata `STRUKTUR` di-*push* ke *stack*, kemudian di-*pop* satu per satu. Karena huruf terakhir keluar lebih dahulu, hasil akhirnya adalah `RUTKURTS`.



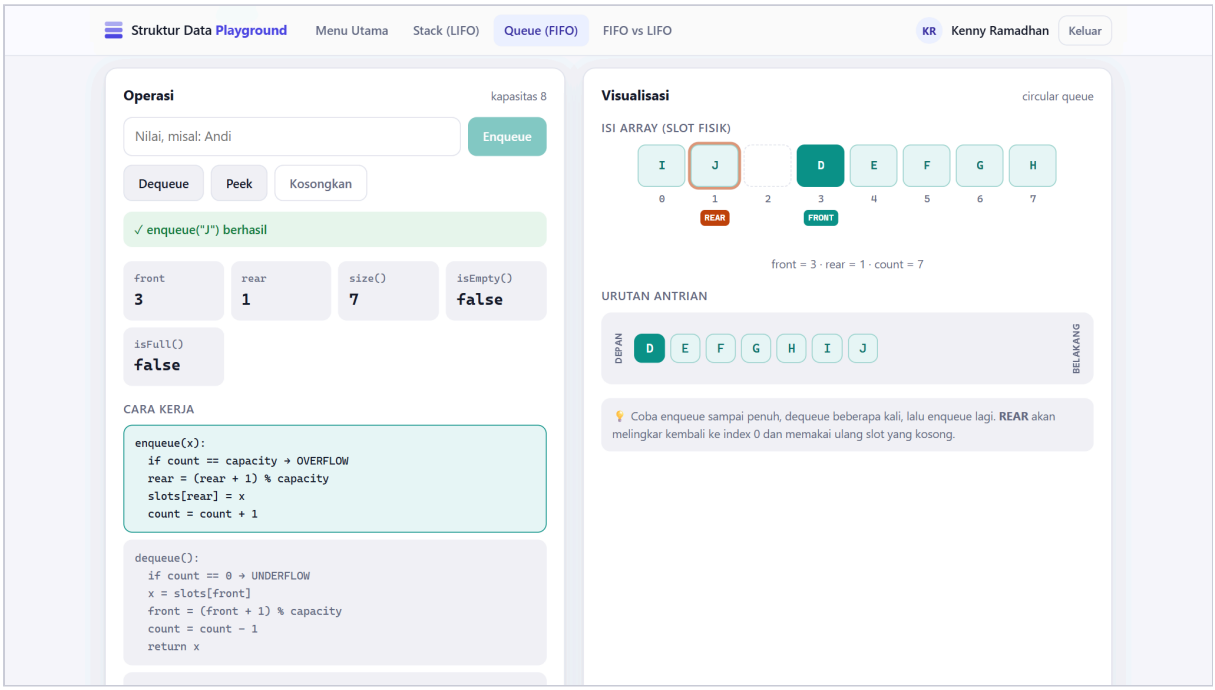
Gambar 4.10 Hasil pembalikan kata menggunakan stack

4.4 Menu Queue (FIFO)

a. Operasi enqueue, dequeue, dan sifat melingkar

Halaman Queue menampilkan dua bentuk visualisasi. **Isi array (slot fisik)** memperlihatkan posisi sebenarnya dari data di dalam *array* beserta penunjuk FRONT dan REAR. **Urutan antrian** memperlihatkan urutan logis data dari depan ke belakang.

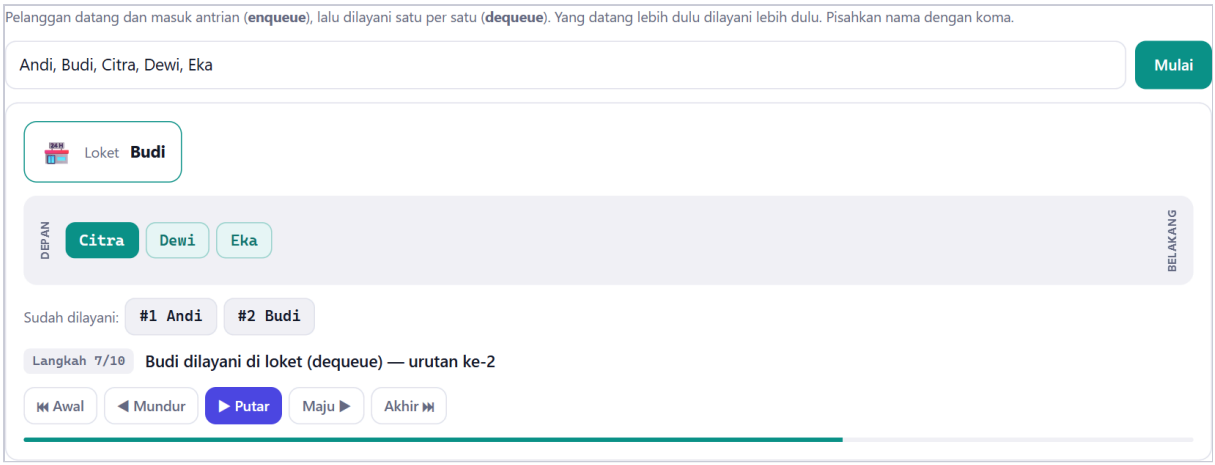
Pada contoh berikut, data A sampai H di-*enqueue* hingga penuh, lalu dilakukan tiga kali *dequeue* sehingga A, B, dan C keluar sesuai urutan masuknya (FIFO). Selanjutnya I dan J di-*enqueue*. Terlihat bahwa penunjuk **REAR melingkar kembali ke index 0 dan 1** dan memakai ulang slot yang sudah kosong, sementara FRONT berada di index 3. Urutan antrian tetap benar, yaitu D, E, F, G, H, I, J.



Gambar 4.11 Circular queue: REAR melingkar ke awal array

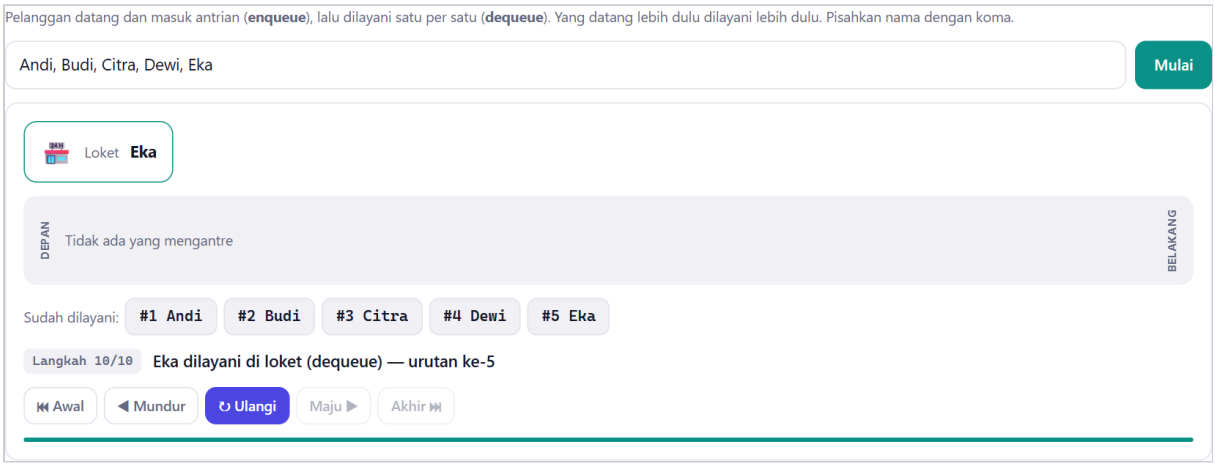
b. Penerapan: simulasi antrian loket

Demo **Simulasi Antrian Loket** menerima daftar nama pelanggan. Seluruh pelanggan terlebih dahulu masuk antrian (*enqueue*), lalu dilayani di loket satu per satu (*dequeue*). Gambar 4.12 memperlihatkan saat Andi (pelanggan pertama) sedang dilayani, sedangkan Budi berada di posisi terdepan antrian.



Gambar 4.12 Proses simulasi antrian loket

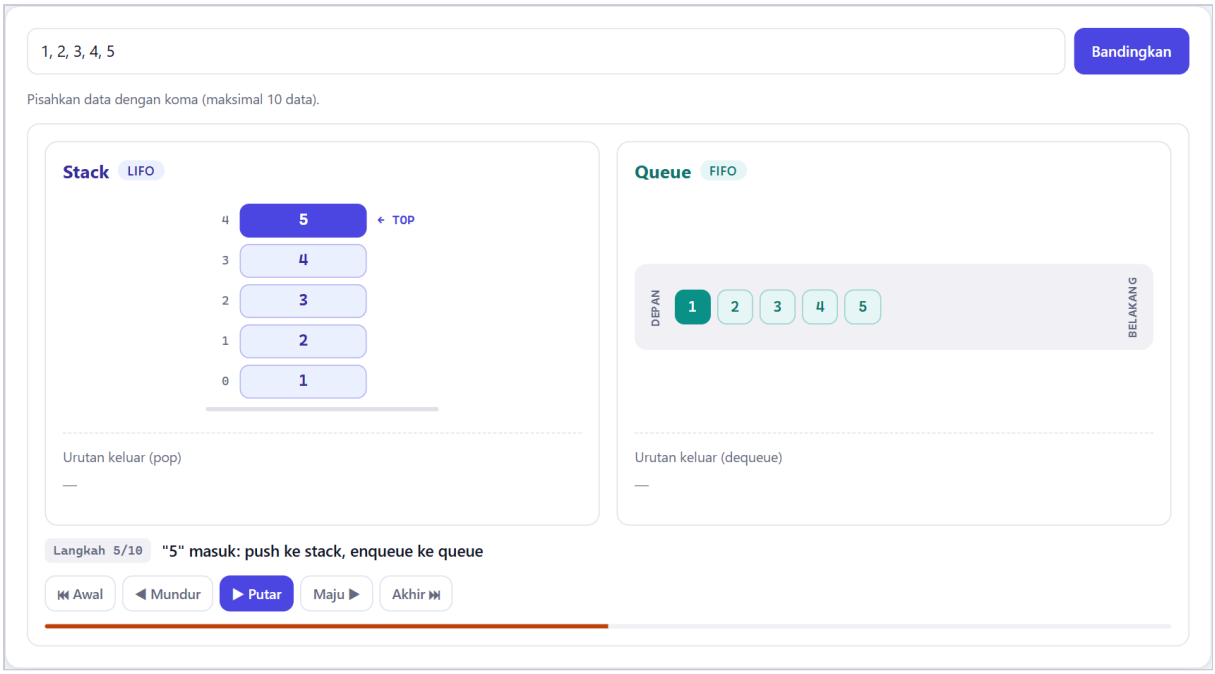
Pada akhir simulasi, seluruh pelanggan dilayani dengan urutan yang sama dengan urutan kedatangannya: Andi, Budi, Citra, Dewi, kemudian Eka.



Gambar 4.13 Hasil akhir simulasi antrian loket

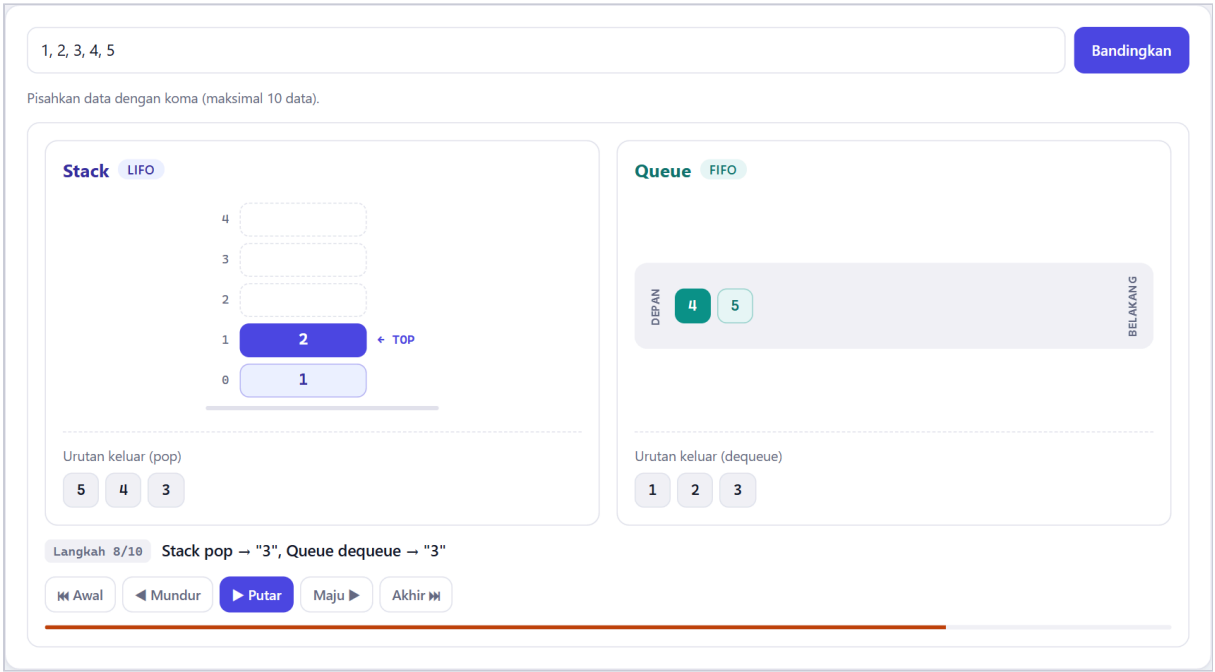
4.5 Menu FIFO vs LIFO

Menu ini membandingkan kedua prinsip secara langsung. Data `1, 2, 3, 4, 5` dimasukkan ke *stack* (*push*) dan ke *queue* (*enqueue*) secara bersamaan.



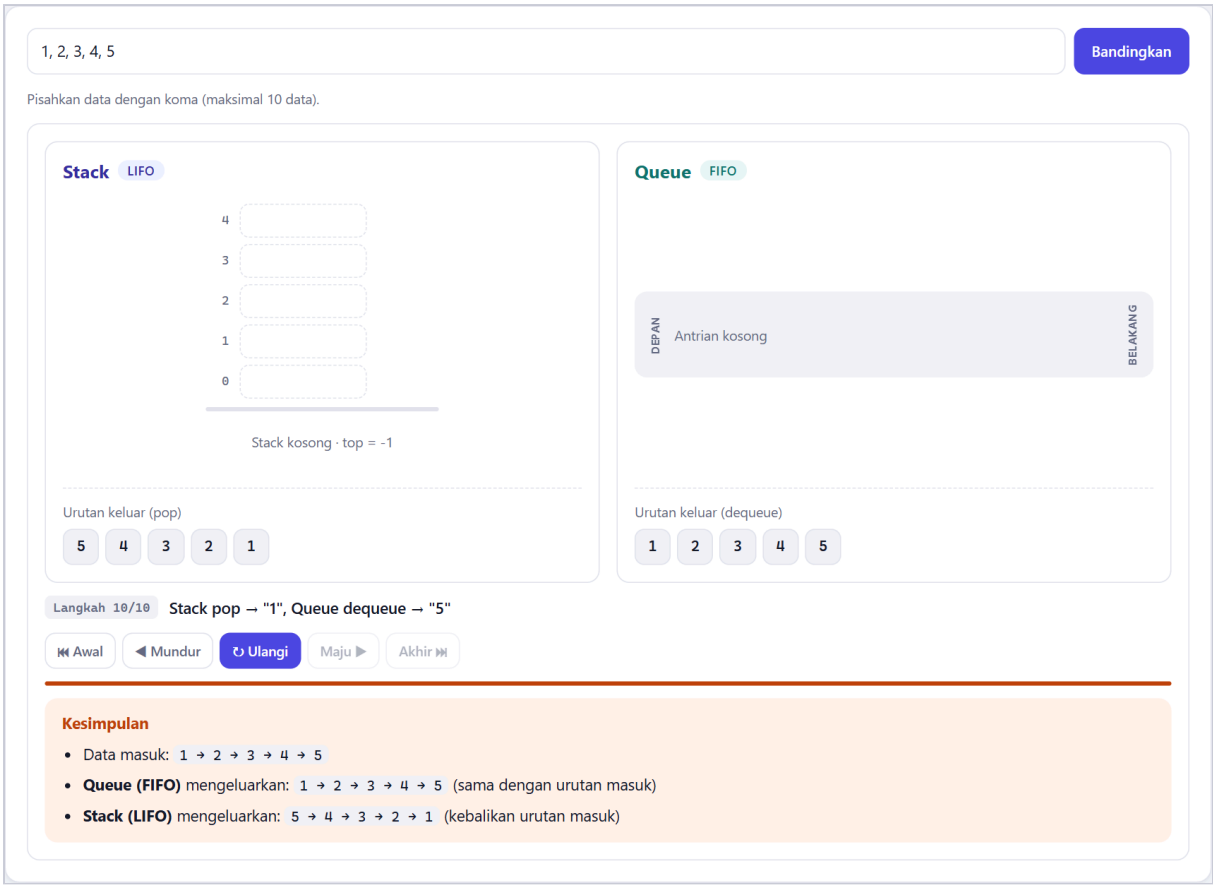
Gambar 4.14 Fase masuk: data dimasukkan ke stack dan queue

Pada fase keluar, *stack* mengeluarkan data dari TOP (5, 4, 3, ...), sedangkan *queue* mengeluarkan data dari FRONT (1, 2, 3, ...).



Gambar 4.15 Fase keluar: urutan keluar stack dan queue berbeda

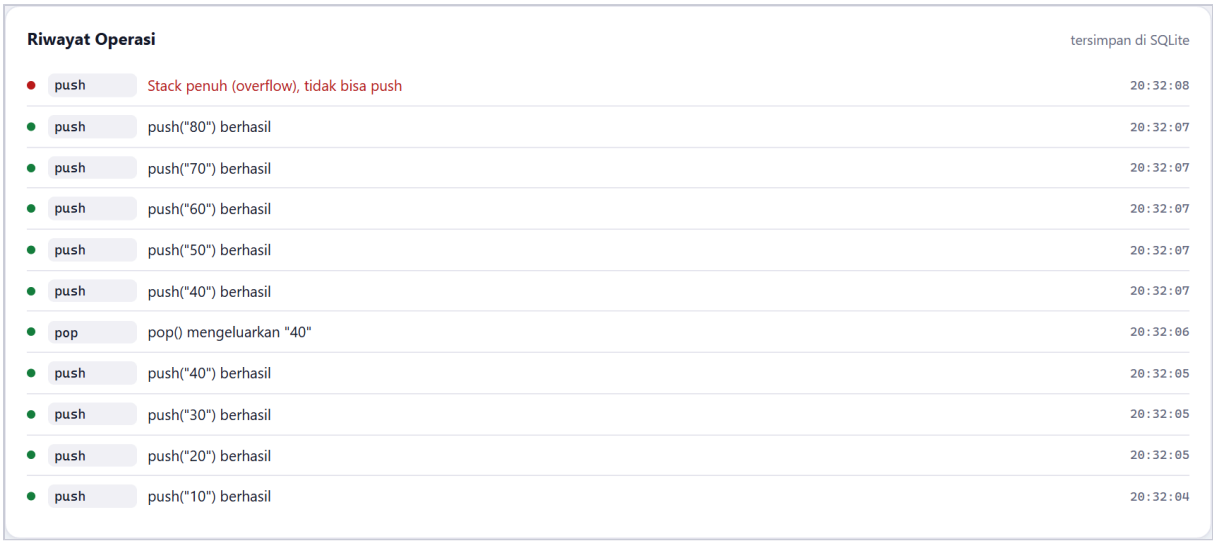
Pada langkah terakhir, aplikasi menampilkan kesimpulan bahwa *queue* (FIFO) mengeluarkan data sesuai urutan masuk, sedangkan *stack* (LIFO) mengeluarkan data dalam urutan terbalik.



Gambar 4.16 Kesimpulan perbandingan FIFO dan LIFO

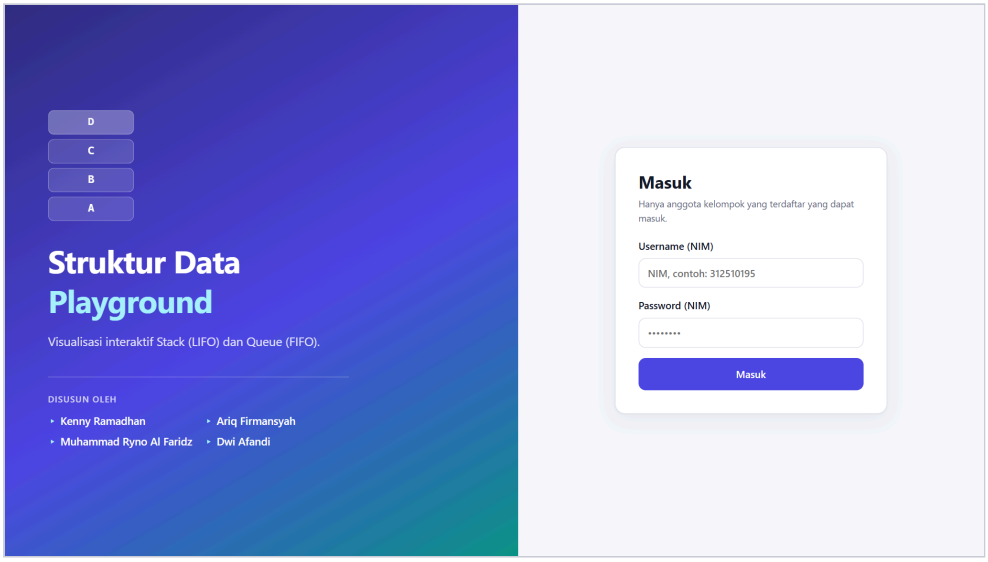
4.6 Riwayat Operasi, Logout, dan Tampilan Responsif

Setiap operasi yang dijalankan pada menu Stack dan Queue dicatat ke tabel `operation_log` dan ditampilkan pada panel **Riwayat Operasi**. Titik hijau menandakan operasi berhasil, sedangkan titik merah menandakan operasi gagal (misalnya *overflow*). Isi *stack* dan *queue* juga tersimpan di basis data, sehingga tidak hilang ketika halaman dimuat ulang, pengguna logout, atau server dijalankan ulang.



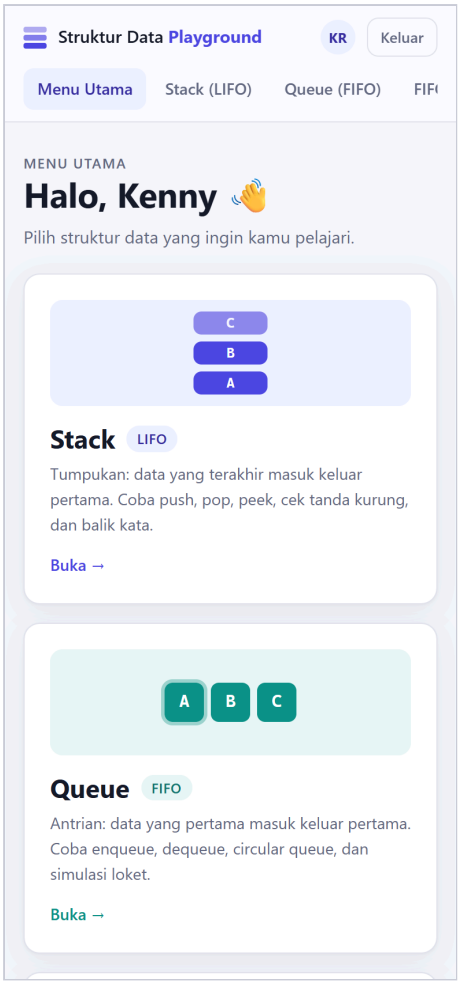
Gambar 4.17 Panel riwayat operasi yang tersimpan di SQLite

Tombol **Keluar** pada bagian kanan atas menghapus token sesi dan mengembalikan pengguna ke halaman login. Halaman lain tidak dapat diakses kembali sebelum login ulang.



Gambar 4.18 Kembali ke halaman login setelah logout

Tampilan aplikasi bersifat responsif sehingga tetap nyaman digunakan pada layar ponsel.



Gambar 4.19 Tampilan menu utama pada layar ponsel

4.7 Pengujian

Pengujian dilakukan dengan dua cara. Pertama, unit test otomatis menggunakan *test runner* bawaan Node.js (`npm test`) untuk memastikan kebenaran logika struktur data. Kedua, pengujian fungsional melalui antarmuka web seperti yang ditunjukkan pada subbab 4.1 hingga 4.6.

Tabel 4.1 Hasil unit test

No	Skenario pengujian	Hasil
1	Stack mengikuti LIFO: data terakhir masuk keluar pertama	Lulus
2	Stack menolak push saat penuh (overflow) dan pop saat kosong (underflow)	Lulus
3	Isi dan urutan stack tetap sama setelah disimpan dan dimuat ulang	Lulus
4	Queue mengikuti FIFO: data pertama masuk keluar pertama	Lulus
5	Queue menolak enqueue saat penuh dan dequeue saat kosong	Lulus
6	Circular queue memakai ulang slot depan yang sudah kosong	Lulus
7	Posisi melingkar queue tetap sama setelah dimuat ulang	Lulus
8	Cek tanda kurung mengenali ekspresi seimbang	Lulus
9	Cek tanda kurung mengenali ekspresi tidak seimbang	Lulus
10	Balik kata menghasilkan urutan terbalik	Lulus
11	Simulasi loket melayani sesuai urutan kedatangan	Lulus
12	Perbandingan: queue keluar sesuai urutan, stack terbalik	Lulus

Keluaran perintah `npm test`:

```
✓ checkBrackets: ekspresi seimbang
✓ checkBrackets: ekspresi tidak seimbang
✓ reverseText membalik kata dengan stack
✓ simulateCounter melayani sesuai urutan datang
✓ compareFifoLifo: urutan keluar queue sama, stack terbalik
✓ Stack: LIFO — data terakhir masuk keluar pertama
✓ Stack: overflow saat penuh dan underflow saat kosong
✓ Stack: toJSON/fromJSON mempertahankan isi dan urutan
✓ Queue: FIFO — data pertama masuk keluar pertama
✓ Queue: overflow saat penuh dan underflow saat kosong
✓ Queue: circular — slot depan yang kosong dipakai ulang
✓ Queue: fromJSON memulihkan posisi melingkar
ℹ tests 12
ℹ pass 12
ℹ fail 0
```

Tabel 4.2 Hasil pengujian fungsional

No	Skenario	Hasil yang diharapkan	Status
1	Login dengan akun yang tidak terdaftar	Ditolak dengan pesan kesalahan	Sesuai
2	Login dengan NIM anggota	Masuk ke Menu Utama	Sesuai
3	Menambah pengguna kelima ke basis data	Ditolak oleh *trigger* "Maksimal 4 user"	Sesuai
4	Membuka halaman tanpa login	Diarahkan ke halaman login	Sesuai
5	Push 9 kali pada stack berkapasitas 8	Push ke-9 ditolak (overflow)	Sesuai
6	Pop pada stack kosong	Ditolak (underflow)	Sesuai
7	Enqueue setelah dequeue pada queue penuh	REAR melingkar ke index 0	Sesuai
8	Memuat ulang halaman / login ulang	Isi stack, queue, dan riwayat tetap ada	Sesuai
9	Menjalankan aplikasi dengan Docker	Aplikasi berjalan dengan `docker compose up`	Sesuai

BAB V
PENUTUP

5.1 Kesimpulan

1. *Stack* berhasil diimplementasikan menggunakan *array* statis dan penunjuk `top`, sedangkan *queue* diimplementasikan sebagai *circular queue* dengan penunjuk `front` dan `rear`. Seluruh operasi berjalan dalam waktu O(1).
2. Aplikasi membuktikan perbedaan prinsip kedua struktur data: *stack* mengeluarkan data dalam urutan terbalik dari urutan masuk (LIFO), sedangkan *queue* mengeluarkan data sesuai urutan masuk (FIFO).
3. *Circular queue* mampu memakai ulang slot yang sudah dikosongkan, sehingga tidak terjadi pemborosan ruang seperti pada *queue* linear.
4. Kondisi *overflow* dan *underflow* berhasil dideteksi dan ditampilkan kepada pengguna.
5. Penerapan *stack* pada pengecekan tanda kurung dan pembalikan kata, serta penerapan *queue* pada simulasi antrian loket, berjalan sesuai teori.
6. Sistem login hanya dapat diakses oleh empat anggota kelompok, dan seluruh data tersimpan di basis data SQLite.

5.2 Saran

1. Menambahkan implementasi *stack* dan *queue* berbasis *linked list* sebagai pembanding implementasi berbasis *array*.
2. Menambahkan varian *queue* lain seperti *priority queue* dan *deque* (*double-ended queue*).
3. Menambahkan penerapan *stack* lainnya, misalnya konversi ekspresi infiks ke postfiks.
4. Memungkinkan pengguna mengatur kapasitas *stack* dan *queue* secara langsung dari antarmuka.

LAMPIRAN

A. Cara Menjalankan Aplikasi

Kode sumber tersedia di repositori **https://github.com/kennyRamadhan/struktur_data**. Unduh atau *clone* repositori tersebut, lalu pilih salah satu cara berikut.

Cara 1: Docker (disarankan)

```
git clone https://github.com/kennyRamadhan/struktur_data.git
cd struktur_data
docker compose up --build
```

Cara 2: Node.js (versi 22.13 atau lebih baru)

```
git clone https://github.com/kennyRamadhan/struktur_data.git
cd struktur_data
npm install
npm start
```

Setelah server berjalan, buka **http://localhost:3000** pada peramban. Unit test dapat dijalankan dengan perintah `npm test`.

> Jika port 3000 sudah digunakan aplikasi lain, jalankan Docker dengan port berbeda, misalnya `APP_PORT=8080 docker compose up --build` (di PowerShell: `$env:APP_PORT=8080; docker compose up --build`), lalu buka http://localhost:8080.

B. Akun Login

Tabel B.1 Akun anggota kelompok

No	Nama	NIM	Username	Password
1	Kenny Ramadhan	312510195	312510195	312510195
2	Ariq Firmansyah	312510176	312510176	312510176
3	Muhammad Ryno Al Faridz	312510022	312510022	312510022
4	Dwi Afandi	312510100	312510100	312510100